DELTA – Střední škola informatiky a ekonomie, s.r.o.
Ke Kamenci 151, Pardubice

LifeMeter

Mobilní aplikace pro sledování zdravotních metrik

Autor: Zadrobílek Jan
Třída: 4.B
Studijní obor: Informační technologie 18-20-M/01
Školní rok: 2025/2026

Zadání maturitního projektu z informatických předmětů

Jméno a příjmení:	*Jan Zadrobílek*
Školní rok:	*2025/2026*
Třída:	*4.B*
Obor:	*Informační technologie 18-20-M/01*
Téma práce:	*Mobilní aplikace „LifeMeter“*
Vedoucí práce:	*Ing. Monika Borkovcová, Ph.D.*

Způsob zpracování, cíle práce, pokyny k obsahu a rozsahu práce:

Cílem projektu je vytvoření mobilní aplikace pro OS Android, která bude sloužit jako nástroj pro sledování fyzické aktivity, nutričních hodnot a spánku. Funkce budou zahrnovat možnost přihlášení, notifikace a lokální manipulaci s daty. Aplikace bude navržena tak, aby poskytovala uživatelům intuitivní uživatelské rozhraní a efektivní způsob interakce. Aplikace je funkční pro OS Android 10 a vyšší.

Popis funkčnosti mobilní aplikace, která umožňuje:

Záznam fyzických aktivit

- Možnost přidávat aktivity s informacemi jako poloha nebo poznámka.

Sledování výživy

- Přidávání zkonzumovaného jídla, doplňků stravy, sledování kalorií, makroživin a vitamínů.

Sledování spánku

- Záznam délky spánku s možností přidání poznámek, případně možnost pracovat s kvalitou spánku.

Správa uživatelů

- Přihlašování bude probíhat pomocí emailu se zabezpečeným ověřením.

Offline

- Aplikace bude umožňovat offline režim se synchronizací dat při obnovení připojení.

Notifikace

- Upozornění ke cvičení a spánku.

Integrace s jinými aplikacemi

- Aplikace bude umožňovat propojení s vybranou aplikací/platformou.

Prohlašuji, že jsem maturitní projekt vypracoval(a) samostatně, výhradně s použitím uvedené literatury.

V Pardubicích 18. 3. 2026 .....

(vlastnoruční podpis)

Rád bych upřímně poděkoval své vedoucí práce, Ing. Monice Borkovcové, za odborné vedení, rady, trpělivost a čas, který mi věnovala při konzultacích a zpracování tohoto maturitního projektu.

Resumé

Tato práce představuje aplikaci „LifeMeter“, určenou pro sledování a analýzu životních metrik uživatele za účelem osobního rozvoje a zlepšení zdraví. Aplikace umožňuje uživatelům centralizované zaznamenávání a monitorování stravy, kvality spánku a fyzické aktivity.

Backendová část systému je postavena na technologiích Bun, Hono a relační databázi PostgreSQL. Pro komunikaci mezi mobilní aplikací a serverem je implementováno REST API, které zajišťuje autentizaci, synchronizaci dat a správu uživatelských profilů.

Mobilní klientská aplikace je vyvinuta pomocí technologií React Native a Expo. Zvolený přístup zaručuje kompatibilitu a zároveň umožňuje přístup k nativním rozhraním pro synchronizaci zdravotních dat, konkrétně HealthKit pro operační systém iOS a Health Connect pro platformu Android.

Klíčová slova: *mobilní aplikace, zdravotní metriky, React Native, Expo, REST API, Bun, Hono, PostgreSQL*

Summary

This thesis presents the „LifeMeter“ application, designed for tracking and analyzing a user's lifestyle metrics to support personal development and health improvement. The application allows users to centrally log and monitor their diet, sleep quality, and physical activity.

The backend of the system is built on Bun, Hono, and a PostgreSQL relational database. A REST API is implemented for communication between the mobile application and the server, providing authentication, data synchronization, and user profile management.

The mobile client application is developed using React Native and Expo. The chosen approach ensures cross-platform compatibility while enabling access to native APIs for health data synchronization, specifically HealthKit for the iOS operating system and Health Connect for Android.

Keywords: *mobile application, health metrics, React Native, Expo, REST API, Bun, Hono, PostgreSQL*

Obsah

Úvod **8**

1 Teoretická část **9**

1.1 Současný stav . . . 9

1.2 Existující řešení . . . 10

1.2.1 Google Fit . . . 10

1.2.2 Apple Health . . . 10

1.2.3 MyFitnessPal . . . 10

1.2.4 Cronometer . . . 10

1.2.5 Sleep Cycle . . . 11

1.3 Volba nástrojů a zdůvodnění . . . 11

1.3.1 React Native a Expo . . . 11

1.3.2 Bun a Hono . . . 11

1.3.3 PostgreSQL . . . 12

1.3.4 Better Auth . . . 12

1.3.5 Docker a Oracle Cloud . . . 12

2 Metodika a vlastní řešení **13**

2.1 Architektura systému . . . 13

2.1.1 Klientská vrstva . . . 13

2.1.2 Serverová vrstva . . . 13

2.1.3 Datová vrstva . . . 15

2.2 Datový model . . . 15

2.2.1 Autentizace . . . 15

2.2.2 Výživa . . . 17

2.2.3 Spánek . . . 17

2.2.4 Tréninky . . . 17

2.2.5 Uživatelský profil . . . 20

2.3 Popis řešení . . . 21

2.3.1 Mobilní aplikace . . . 21

2.3.2 Serverová část 22
2.3.3 CI/CD pipeline 24
2.4 Návrh aplikace 24
2.4.1 Případy užití 24
2.4.2 Diagram navigace 24
2.4.3 Diagram tříd 25
2.5 Popis důležitých algoritmů 27
2.5.1 Výpočet metabolismu a denního energetického výdeje 27
2.5.2 Stanovení cílového denního kalorického příjmu 28
2.5.3 Výpočet nutričních hodnot jídla 28
2.5.4 Fulltextové vyhledávání potravin 29
2.5.5 Offline fronta požadavků 30

Výsledky **32**

Diskuze **33**

Závěr **34**

Literatura **35**

Seznam obrázků **36**

Seznam tabulek **37**

Seznam příloh **39**

A: Diagramy
B: Tabulky
C: Schémata
D: API dokumentace
E: Instalační návod

Úvod

Sledování zdravotních metrik se v posledních letech stalo běžnou součástí každodenního života stále většího počtu lidí. Rozšíření chytrých telefonů a nositelných zařízení umožnilo sbírat data o fyzické aktivitě, výživě či spánku způsobem, který byl dříve dostupný pouze ve specializovaných zařízeních. Zájem o oblast takzvaného *quantified self* – tedy systematického zaznamenávání vlastních biologických a behaviorálních dat za účelem osobního rozvoje – neustále roste [1].

Stávající řešení na trhu (např. MyFitnessPal, Google Fit či Hevy) se často zaměřují jen na dílčí aspekty zdraví a uživatel je nucen přecházet mezi více nepropojenými aplikacemi, mnohdy s omezenými funkcemi v neplacených verzích. Cílem tohoto maturitního projektu je proto návrh a implementace aplikace *LifeMeter*, která usiluje o centralizaci sledovaných metrik (fyzické aktivity, výživa, spánek, hmotnost) do jediného prostředí s přehlednou domovskou obrazovkou.

Mobilní klient pro systémy Android a iOS je vyvinut pomocí frameworků React Native [2] a Expo [3]. Synchronizaci dat ze systémových úložišť lokálních zařízení zajišťují rozhraní Health Connect a HealthKit. Serverová část, komunikující s klientem přes REST API, je postavena na moderním běhovém prostředí Bun [4] s využitím frameworku Hono a relační databáze PostgreSQL. Celý backendový systém je kontejnerizován v prostředí Docker a nasazen na platformě Oracle Cloud.

1. Teoretická část

1.1 Současný stav

Moderní mobilní operační systémy poskytují vývojářům třetích stran standardizovaná rozhraní pro přístup ke zdravotním datům uloženým v centralizovaném systémovém úložišti. Na platformě Android tuto roli od verze Android 8 zastává Health Connect [5], na platformě iOS pak HealthKit [6], dostupný od verze iOS 8. Obě rozhraní fungují na principu zprostředkování – aplikace nezíská přímý přístup k datům jiných aplikací, nýbrž čte z a zapisuje do sdíleného úložiště, přičemž uživatel musí explicitně udělit oprávnění ke každému datovému typu zvlášť. Tento model výrazně zjednodušuje integraci mezi aplikacemi (sledování kroků, chytré hodinky, fitness tracker) a eliminuje nutnost vytvářet proprietární synchronizační protokoly.

Klíčovým modelem při návrhu mobilních aplikací závislých na vzdáleném serveru je přístup *offline-first* [7]. Mobilní zařízení jsou ze své podstaty vystavena nestabilnímu nebo zcela chybějícímu síťovému připojení, proto robustní aplikace nepředpokládá nepřetržitou konektivitu, nýbrž primárně pracuje s lokálně uloženými daty a se serverem se synchronizuje, jakmile je připojení k dispozici. Na platformě React Native je standardním řešením pro rychlé lokální úložiště knihovna MMKV [8], která využívá nativní implementaci ve formě sdílené paměti mapované na soubor a dosahuje výrazně nižší latence než `AsyncStorage`.

Komunikace mezi mobilním klientem a serverovou částí je v současných aplikacích zpravidla realizována prostřednictvím architektury REST (*Representational State Transfer*) [9]. REST definuje sadu architektonických omezení – bezstavovost, jednotné rozhraní, vrstvený systém – která vedou k interoperabilním, škálovatelným a snadno testovatelným API. Každý zdroj je identifikován URL adresou, operace nad ním jsou vyjádřeny standardními HTTP metodami (GET, POST, PUT, DELETE) a data jsou přenášena zpravidla ve formátu JSON.

Autentizace uživatelů v moderních webových a mobilních aplikacích se nejčastěji řeší dvěma přístupy. Prvním jsou tokeny JWT (*JSON Web Token*), které jsou bezstavové a snadno horizontálně škálovatelné, avšak jejich uložení na straně klienta přináší bezpečnostní rizika, pokud jsou uchovávány v přístupném úložišti. Druhým přístupem jsou

stavové relace spravované serverem s identifikátorem uloženým v `HttpOnly` cookie, která není přístupná JavaScriptovým kódem, čímž je eliminováno riziko krádeže tokenu skriptem třetí strany [10].

1.2 Existující řešení

Na trhu existuje řada aplikací zaměřených na sledování zdravotních metrik. Níže jsou popsána nejrozšířenější řešení včetně jejich omezení, jež vedla k rozhodnutí vytvořit vlastní aplikaci.

1.2.1 Google Fit

Google Fit [11] je platforma vyvinutá společností Google, integrovaná přímo do operačního systému Android. Automaticky zaznamenává pohybovou aktivitu prostřednictvím senzorů zařízení a slouží jako centrální úložiště zdravotních dat pro ekosystém Android. Aplikace nabízí jednoduché přehledy a cíle, avšak detailní sledování výživy ani spánku nativně nepodporuje – tyto oblasti vyžadují napojení na specializované aplikace třetích stran.

1.2.2 Apple Health

Apple Health [12] zastává v ekosystému iOS analogickou roli jako Google Fit na Androidu – jde o systémové úložiště zdravotních dat přístupné přes rozhraní HealthKit. Aplikace agreguje záznamy z různých zdrojů a zobrazuje je v přehledném rozhraní. Platforma je dostupná výhradně na zařízeních Apple a funguje primárně jako datový hub; vlastní zadávání výživy nebo tréninkových plánů není nativně podporováno.

1.2.3 MyFitnessPal

MyFitnessPal [13] je jednou z nejrozšířenějších aplikací pro sledování výživy na světě. Disponuje rozsáhlou databází potravin, podporou skenování čárových kódů a integrací s fitness zařízeními. Sledování tréninkového zatížení je omezeno na kalorický výdej bez možnosti podrobné analýzy silových tréninků. Velká část funkcionalit – pokročilé makroanalýzy, plánování, hloubkové statistiky – je dostupná pouze v placené verzi Premium.

1.2.4 Cronometer

Cronometer [14] se od konkurence odlišuje důrazem na nutriční přesnost. Databáze potravin vychází primárně z ověřených vědeckých zdrojů (USDA, NCCDB) a aplikace sleduje vedle makroživin také mikroživiny, vitamíny a minerály. Sledování fyzické aktivity

a spánku je v základní verzi omezeno; pokročilé funkce jsou vázány na placenou verzi Gold.

1.2.5 Sleep Cycle

Sleep Cycle [15] je aplikace specializovaná výhradně na analýzu kvality spánku. Ke sledování využívá mikrofon zařízení nebo propojení s chytrými hodinkami a na základě detekce pohybu a zvukových vzorů odhaduje spánkové fáze. Nabízí chytrý budík probouzející uživatele v nejlehčí fázi spánku. Aplikace nenabízí žádnou integraci se sledováním výživy či pohybu a pokročilé statistiky jsou dostupné pouze v rámci předplatného.

1.3 Volba nástrojů a zdůvodnění

Výběr technologií byl proveden s ohledem na požadavky projektu, a osobní znalostí s některými z technologií.

1.3.1 React Native a Expo

Pro vývoj mobilního částí byl zvolen framework React Native [2] v kombinaci s platformou Expo [3]. React Native umožňuje sestavit jedinou sdílenou kódovou základnu v jazyce JavaScript/TypeScript, ze které jsou generovány nativní aplikace pro Android i iOS. Tato vlastnost byla klíčovým kritériem výběru – projekt byl sice primárně navrhován pro Android, avšak podpora iOS je zachována pro případnou budoucí expanzi bez nutnosti přepisovat aplikaci od základu. Rozhodujícím faktorem byla také předchozí znalost ekosystému React, jež výrazně zkrátila dobu potřebnou k osvojení frameworku.

Jako alternativy připadaly v úvahu framework Flutter a nativní vývoj v jazycích Kotlin resp. Swift. Flutter sice nabízí srovnatelnou multiplatformnost, avšak vyžaduje znalost jazyka Dart, který nebylo možné sdílet se serverovou částí. Nativní vývoj by poskytl maximální přístup k systémovým API za cenu nutnosti udržovat dvě zcela oddělené kódové základny.

Expo bylo zvoleno jako současný standard React Native vývoje. Managed workflow zjednodušuje konfiguraci nativních závislostí, nástroj EAS Build zajišťuje automatizované sestavení v rámci CI/CD pipeline na GitHub Actions a Expo SDK poskytuje stabilní přístup k nativním modulům – včetně integrace s rozhraními Health Connect [5] a HealthKit [6].

1.3.2 Bun a Hono

Serverová část je postavena na rozhraní Bun [4] s využitím webového frameworku Hono [16]. Bun je moderní JavaScriptový runtime s výrazně vyšším výkonem oproti Node.js – v po-

rovnání s Node.js dosahuje násobně kratších dob startu i vyšší propustnosti HTTP požadavků [17]. Klíčovou výhodou je vestavěná podpora jazyka TypeScript bez nutnosti transpilace, což zjednodušuje vývojový proces při zachování plné kompatibility s ekosystémem npm.

Hono je odlehčený webový framework optimalizovaný pro serverová prostředí. Vyznačuje se minimálními závislostmi, typově bezpečným směrováním požadavků a přímočarým API. Kombinace Bun + Hono byla zvolena jako moderní alternativa ke standardnímu stacku Node.js + Express.

1.3.3 PostgreSQL

Jako databázový systém byla zvolena databáze PostgreSQL [18]. Datový model aplikace LifeMeter zahrnuje vzájemně provázané entity (uživatelé, jídla, tréninky, spánkové záznamy), jejichž vztahy jsou přirozeně vyjádřeny relačním schématem s cizími klíči a referenční integritou. Relační databáze představuje přirozenější volbu oproti dokumentovým databázím (MongoDB) nebo embedded řešením (SQLite), která by nebyla vhodná pro serverovou část s souběžnými přístupy více uživatelů.

1.3.4 Better Auth

Pro správu autentizace a uživatelských relací byla zvolena knihovna Better Auth [19]. Tato volba byla upřednostněna před vlastní implementací i před cloudovými řešeními typu Firebase Authentication. Better Auth je navržen s nativní podporou jazyka TypeScript a poskytuje přímočarou integraci s frameworkem Hono prostřednictvím dedikovaného adaptéru. Jako self-hosted řešení nevyžaduje závislost na infrastruktuře třetí strany, čímž je zajištěna plná kontrola nad uživatelskými daty. Autentizace je realizována prostřednictvím `HttpOnly` cookies se stavovými relacemi na serveru, v souladu s doporučeními OWASP [10].

1.3.5 Docker a Oracle Cloud

Celý backendový systém je kontejnerizován pomocí nástroje Docker Compose [20] a nasazen na instanci Oracle Cloud Infrastructure. Docker Compose umožňuje deklarativní definici vícekomponentního prostředí (API server, databáze, reverzní proxy) v jediném konfiguračním souboru, čímž zajišťuje reprodukovatelnost nasazení a jednoduchou správu závislostí mezi službami.

Oracle Cloud byl zvolen díky programu Always Free, který zahrnuje bezplatnou instanci s architekturou ARM s dostatečnými výpočetními prostředky pro provoz projektu. Na rozdíl od PaaS řešení poskytuje vlastní virtuální stroj plnou kontrolu nad konfigurací prostředí a správou uložených dat.

2. Metodika a vlastní řešení

2.1 Architektura systému

Systém LifeMeter se skládá ze dvou hlavních součástí: mobilního klienta a serverové části nasazené v cloudovém prostředí. Mobilní klient komunikuje se serverem za použití REST API přes protokol HTTPS. Celkové uspořádání systému je znázorněno na obrázku 2.1.

2.1.1 Klientská vrstva

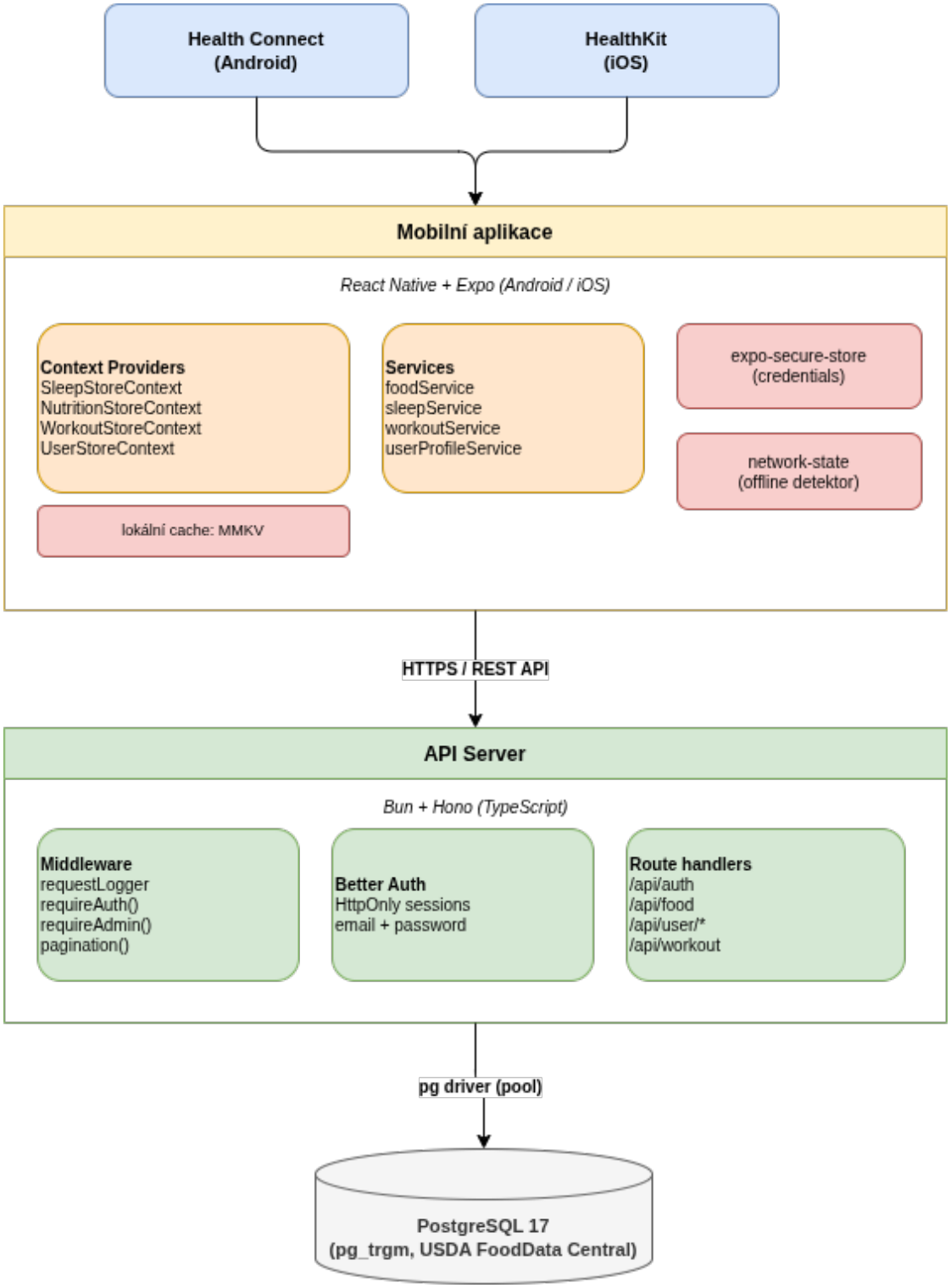
Mobilní aplikace je vyvinuta pomocí frameworku React Native ve spojení s platformou Expo [3], která zajišťuje přenositelnost mezi operačními systémy Android a iOS z jedné sdílené kódové základny. Na platformě Android jsou zdravotní data čtena přes rozhraní Health Connect [5], na platformě iOS pak prostřednictvím HealthKit [6].

Aplikační stav je spravován čtveřicí doménových Context Providers – `SleepStoreContext`, `NutritionStoreContext`, `WorkoutStoreContext` a `UserStoreContext`. Každý provider zapouzdřuje data a operace pro příslušnou doménu a vystavuje je potomkům stromu komponent prostřednictvím React Context API. Lokální perzistenci dat zajišťuje knihovna MMKV [8]: data jsou primárně čtena z cache v paměti zařízení a synchronizována se serverem při obnovení síťového připojení, v souladu s principem *offline-first* [7]. Přihlašovací údaje jsou ukládány šifrovaně prostřednictvím `expo-secure-store`.

2.1.2 Serverová vrstva

Serverová část je implementována v prostředí Bun s využitím webového frameworku Hono [16]. Každý příchozí požadavek prochází globálním middlewarovým řetězcem: `requestLogger` zaznamenává HTTP provoz, autentizační middleware injektuje informace o relaci (`c.get("user")`) a `pagination()` normalizuje parametry stránkování. Správu uživatelských relací zajišťuje knihovna Better Auth [19] prostřednictvím stavových `HttpOnly` cookies, v souladu s doporučeními OWASP [10].

Routy jsou organizovány doménově: `/api/auth` zpracovává přihlášení a registraci, `/api/food` poskytuje přístup k databázi potravin, `/api/user/*` sdružuje endpointy pro spánek, jídla, tréninky a uživatelský profil a `/api/workout` vystavuje referenční data



Obrázek 2.1: Přehled architektury systému LifeMeter

(cviky, styly sérií). Chráněné routy vyžadují middleware `requireAuth()`, administrátorské pak `requireAdmin()`.

2.1.3 Datová vrstva

Datová vrstva je tvořena relační databází PostgreSQL 17 [18]. Rozšíření `pg_trgm` [21] umožňuje efektivní trigram-based fulltextové vyhledávání v databázi potravin, jejíž obsah pochází z veřejně dostupného zdroje USDA FoodData Central [22]. Serverová část komunikuje s databází prostřednictvím connection poolu knihovny `pg`.

2.2 Datový model

Databázové schéma aplikace LifeMeter je rozdělena do pěti doménových skupin: autentizace a správa relací (spravováno knihovnou Better Auth), výživa, spánek, tréninky a uživatelský profil. Schéma je implementováno v PostgreSQL 17 a je verzováno prostřednictvím migračních SQL souborů. Entity-relationship diagram celého schématu je uveden na obrázku 2.2.

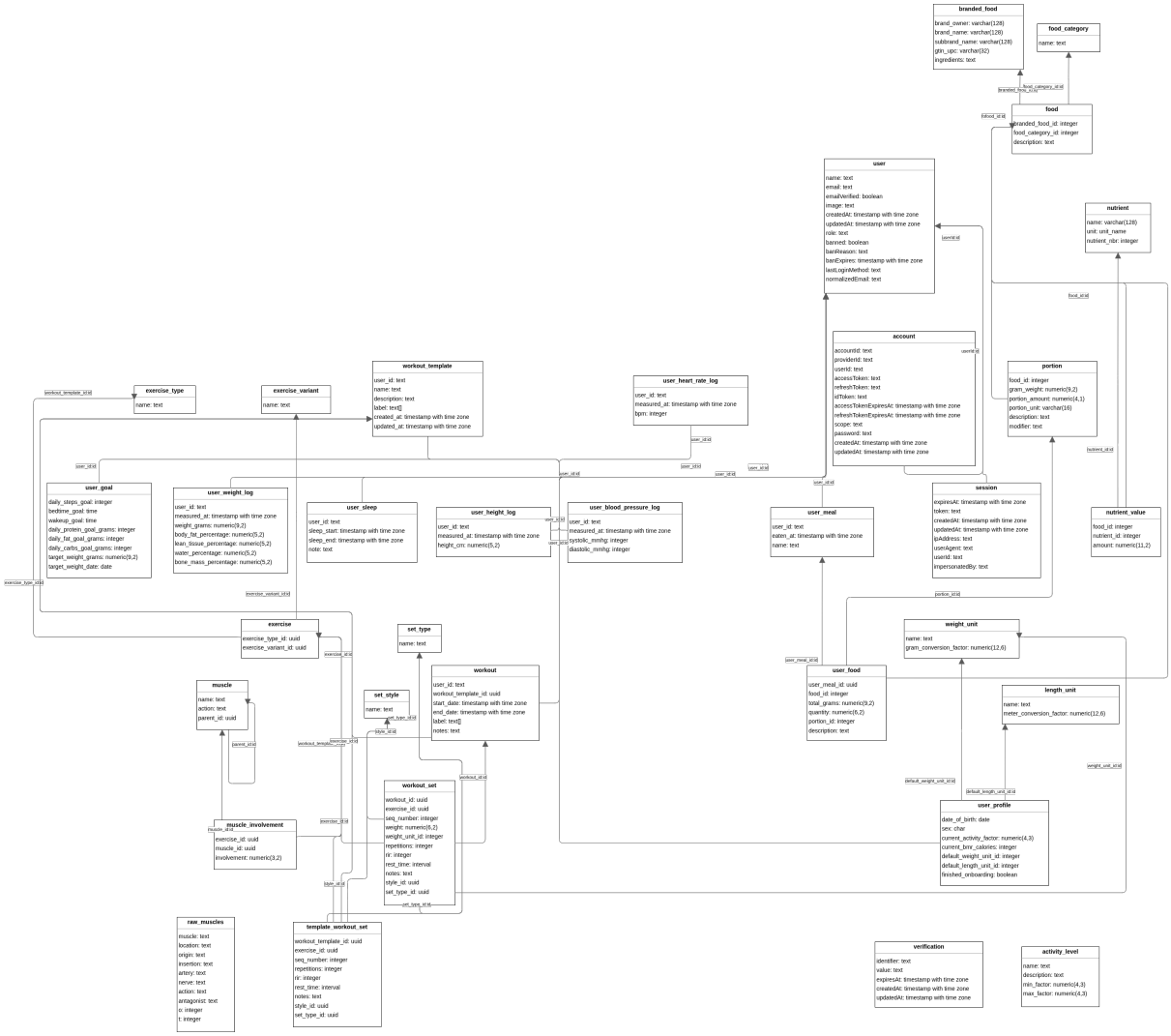
2.2.1 Autentizace

Tabulky pro správu uživatelů, relací a ověřování jsou generovány knihovnou Better Auth [19] a nejsou spravovány vlastními migracemi. Centrální entitou je tabulka `user`, na kterou jsou pomocí cizích klíčů navázány všechny uživatelské záznamy v ostatních doménách.

Tabulka 2.1: Tabulka `user` – uživatelský účet

Sloupec	**Datový typ**	**Popis**
`id`	`TEXT PK`	Unikátní identifikátor uživatele
`name`	`TEXT`	Zobrazované jméno
`email`	`TEXT UNIQUE`	E-mailová adresa (přihlašovací jméno)
`emailVerified`	`BOOLEAN`	Příznak ověření e-mailové adresy
`role`	`TEXT`	Role uživatele (`user` / `admin`)
`banned`	`BOOLEAN`	Příznak zablokování účtu
`createdAt`	`TIMESTAMPTZ`	Datum a čas registrace
`updatedAt`	`TIMESTAMPTZ`	Datum a čas poslední změny

Dále jsou generovány tabulky `session` (aktivní relace s tokenem a IP adresou), `account` (přihlašovací metoda a přihlašovací údaje) a `verification` (jednorázové ověřovací tokeny).



Obrázek 2.2: Entity-relationship diagram databázového schématu

2.2.2 Výživa

Skupina výživových tabulek se skládá ze dvou částí: referenčních dat importovaných z databáze USDA FoodData Central [22] a uživatelských záznamů o konzumovaných jídlech.

Tabulka 2.2: Tabulka `food` – potravina

Sloupec	Datový typ	Popis
`id`	`SERIAL PK`	Identifikátor potraviny
`description`	`TEXT`	Název potraviny; indexován přes `pg_trgm`
`food_category_id`	`INT FK`	Odkaz na kategorii potraviny
`branded_food_id`	`INT FK UNIQUE`	Odkaz na značkové potraviny (nullable)

Tabulka 2.3: Tabulka `nutrient_value` – nutriční hodnota potraviny

Sloupec	Datový typ	Popis
`id`	`SERIAL PK`	Identifikátor záznamu
`food_id`	`INT FK`	Odkaz na potravinu
`nutrient_id`	`INT FK`	Odkaz na nutrient (bílkoviny, tuky, ...)
`amount`	`NUMERIC(11,2)`	Množství nutientu na 100 g potraviny

Tabulka 2.4: Tabulka `portion` – porce potraviny

Sloupec	Datový typ	Popis
`id`	`SERIAL PK`	Identifikátor porce
`food_id`	`INT FK`	Odkaz na potravinu
`gram_weight`	`NUMERIC(9,2)`	Hmotnost porce v gramech
`portion_amount`	`NUMERIC(4,1)`	Počet jednotek porce (nullable)
`portion_unit`	`VARCHAR(16)`	Jednotka porce (např. `cup`, `tbsp`)
`description`	`TEXT`	Slovní popis porce

2.2.3 Spánek

Doména spánku zahrnuje tabulku pro záznam denního spánku uživatele.

2.2.4 Tréninky

Doména tréninků se skládá z referenčních dat (typy cviků, svaly, styly a typy sérií) a uživatelských dat (šablony tréninků a zaznamenané tréninky).

Tabulka 2.5: Tabulka `user_meal` – jídlo uživatele

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor záznamu
`user_id`	`TEXT FK`	Odkaz na uživatele
`eaten_at`	`TIMESTAMPTZ`	Čas konzumace
`name`	`TEXT`	Volitelný název jídla

Tabulka 2.6: Tabulka `user_food` – položka jídla

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor záznamu
`user_meal_id`	`UUID FK`	Odkaz na jídlo (CASCADE DELETE)
`food_id`	`INT FK`	Odkaz na potravinu z databáze
`total_grams`	`NUMERIC(9,2)`	Celková hmotnost konzumované potraviny
`quantity`	`NUMERIC(6,2)`	Počet porcí (výchozí hodnota 1)
`portion_id`	`INT FK`	Odkaz na zvolenou porci (nullable)

Tabulka 2.7: Tabulka `user_sleep` – spánková seance

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor záznamu
`user_id`	`TEXT FK`	Odkaz na uživatele (CASCADE DELETE)
`sleep_start`	`TIMESTAMPTZ`	Čas začátku spánku
`sleep_end`	`TIMESTAMPTZ`	Čas konce spánku (nullable – aktivní seance)
`note`	`TEXT`	Volitelná poznámka

Tabulka 2.8: Tabulka `workout_template` – šablona tréninku

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor šablony
`user_id`	`TEXT FK`	Odkaz na uživatele
`name`	`TEXT`	Název šablony
`description`	`TEXT`	Volitelný popis
`label`	`TEXT[]`	Pole štítků (např. `{Push, Hypertrophy}`)
`created_at`	`TIMESTAMPTZ`	Datum vytvoření
`updated_at`	`TIMESTAMPTZ`	Datum poslední změny

Tabulka 2.9: Tabulka `template_workout_set` – série v šabloně tréninku

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor série
`workout_template_id`	`UUID FK`	Odkaz na šablonu (CASCADE DELETE)
`exercise_id`	`UUID FK`	Odkaz na cvik
`seq_number`	`INT`	Pořadí série v šabloně
`repetitions`	`INT`	Plánovaný počet opakování
`rir`	`INT`	Reps in reserve – počet rezervních opakování
`rest_time`	`INTERVAL`	Délka odpočinku
`style_id`	`UUID FK`	Odkaz na styl série (nullable)
`set_type_id`	`UUID FK`	Odkaz na typ série (nullable)

Tabulka 2.10: Tabulka `workout` – zaznamenaný trénink

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor tréninku
`user_id`	`TEXT FK`	Odkaz na uživatele
`workout_template_id`	`UUID FK`	Odkaz na použitou šablonu (nullable)
`start_date`	`TIMESTAMPTZ`	Čas začátku tréninku
`end_date`	`TIMESTAMPTZ`	Čas konce tréninku (nullable)
`label`	`TEXT[]`	Pole štítků
`notes`	`TEXT`	Volitelné poznámky

Tabulka 2.11: Tabulka `workout_set` – zaznamenaná série tréninku

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor série
`workout_id`	`UUID FK`	Odkaz na trénink
`exercise_id`	`UUID FK`	Odkaz na cvik
`seq_number`	`INT`	Pořadí série v tréninku
`weight`	`NUMERIC(6,2)`	Použitá zátěž (nullable)
`weight_unit_id`	`INT FK`	Jednotka zátěže (nullable)
`repetitions`	`INT`	Skutečný počet opakování
`rir`	`INT`	Skutečný počet rezervních opakování (nullable)
`style_id`	`UUID FK`	Styl série (nullable)
`set_type_id`	`UUID FK`	Typ série (nullable)

Referenční tabulky `exercise` (kombinace typu a varianty cviku), `exercise_type`, `exercise_variant`, `muscle` (hierarchická tabulka svalů s vlastní rekurzivní vazbou `parent_id`), `muscle_involvement` (míra zapojení svalu v cviku), `set_style` a `set_type` obsahují pouze identifikátor a název, případně doplňující číselné atributy, a slouží jako číselníky.

2.2.5 Uživatelský profil

Tabulka 2.12: Tabulka `user_profile` – profil uživatele

Sloupec	Datový typ	Popis
`user_id`	`TEXT PK FK`	Odkaz na uživatele (1:1)
`date_of_birth`	`DATE`	Datum narození
`sex`	`CHAR(1)`	Pohlaví (`M` / `F`)
`current_activity_factor`	`NUMERIC(4,3)`	Aktuální faktor aktivity (výchozí 1,200)
`current_bmr_calories`	`INT`	Bazální metabolismus v kcal
`default_weight_unit_id`	`INT FK`	Výchozí jednotka hmotnosti
`default_length_unit_id`	`INT FK`	Výchozí jednotka délky
`finished_onboarding`	`BOOLEAN`	Příznak dokončení úvodního průvodce

Tabulka 2.13: Tabulka `user_goal` – cíle uživatele

Sloupec	Datový typ	Popis
`user_id`	`TEXT PK FK`	Odkaz na uživatele (1:1)
`daily_steps_goal`	`INT`	Denní cíl kroků
`bedtime_goal`	`TIME`	Cílový čas ulehnutí
`wakeup_goal`	`TIME`	Cílový čas probuzení
`daily_protein_goal_grams`	`INT`	Denní cíl bílkovin v g
`daily_fat_goal_grams`	`INT`	Denní cíl tuků v g
`daily_carbs_goal_grams`	`INT`	Denní cíl sacharidů v g
`target_weight_grams`	`NUMERIC(9,2)`	Cílová hmotnost v gramech
`target_weight_date`	`DATE`	Datum dosažení cílové hmotnosti

Dále jsou evidovány tabulky `user_height_log` (výška v cm s časovým razítkem), `user_blood_pressure_log` (systolický a diastolický tlak v mmHg) a `user_heart_rate_log` (tepová frekvence v bpm). Všechny protokolové tabulky sdílejí společný vzor: primární klíč `UUID`, cizí klíč `user_id` s `CASCADE DELETE` a časové razítko `measured_at` indexované sestupně pro efektivní výpis posledních hodnot.

Tabulka 2.14: Tabulka `user_weight_log` – záznam hmotnosti

Sloupec	Datový typ	Popis
`id`	`UUID PK`	Identifikátor záznamu
`user_id`	`TEXT FK`	Odkaz na uživatele (CASCADE DELETE)
`measured_at`	`TIMESTAMPTZ`	Datum a čas měření
`weight_grams`	`NUMERIC(9,2)`	Naměřená hmotnost v gramech
`body_fat_percentage`	`NUMERIC(5,2)`	Procento tělesného tuku (nullable)
`lean_tissue_percentage`	`NUMERIC(5,2)`	Procento svalové hmoty (nullable)
`water_percentage`	`NUMERIC(5,2)`	Procento vody v těle (nullable)
`bone_mass_percentage`	`NUMERIC(5,2)`	Procento kostní hmoty (nullable)

2.3 Popis řešení

2.3.1 Mobilní aplikace

Autentizace a správa relací

Autentizace je realizována prostřednictvím `AuthProvider` – kontextu obalujícího celý strom aplikace. Při inicializaci se volá `authService.getSession()`, jehož výsledek je uložen v zabezpečeném úložišti `expo-secure-store` pod klíčem `auth-session-cache`. Při selhání síťového volání je relace obnovena z tohoto lokálního záznamu, čímž je zachována funkčnost i při výpadku připojení. Přihlašování a registrace probíhá voláním REST endpointů Better Auth (`/api/auth/sign-in/email`, `/api/auth/sign-up/email`); stavová `HttpOnly` cookie je spravována serverem a automaticky přikládána ke každému požadavku prostřednictvím volby `credentials: "include"`.

Modul zdravotních dat

Čtení zdravotních dat ze systémových úložišť je zapouzdřeno do abstrakční vrstvy `src/lib/health/`, která sjednocuje rozhraní Health Connect [5] na platformě Android a HealthKit [6] na platformě iOS do jediného platformově neutrálního API. Každá funkce (například `getSteps`, `getSleep`, `getWeight`) vrací strukturu `Result<T>` rozlišující úspěšný výsledek a typovanou chybu, čímž je zamezeno nekontrolovanému šíření výjimek do vyšších vrstev.

Oprávnění jsou vyžádána jednorázově při startu aplikace funkcí `requestHealthPermissions()`. Na platformě Android je inicializován klient Health Connect a pro každý typ dat (`Steps`, `SleepSession`, `Weight` aj.) je vyžádáno oprávnění ke čtení; na platformě iOS je volána metoda `requestAuthorization()` knihovny HealthKit. Hodnoty spánkových fází jsou mapovány z číselných konstant příslušné platformy na sjednocenou množinu `"awake"| "core"| "deep"| "rem"| "asleep"`.

Lokální úložiště

Lokální perzistence dat je realizována modulem `src/lib/storage.ts`, který obaluje MMKV [8] typově bezpečnými pomocnými funkcemi pro čtení a zápis primitiv, objektů a polí serializovaných do JSON. Pro použití uvnitř komponent React jsou exportovány hooky `useStorage.string()`, `useStorage.boolean()`, `useStorage.object()` a `useStorage.array()`, jejichž změna hodnoty způsobí překreslení komponenty v souladu s modelem React. Pro imperativní přístup mimo komponenty (například ve frontě offline požadavků) slouží synchronní API objektu `storage`.

Offline fronta požadavků

Síťová vrstva aplikace (`src/lib/net.ts`) implementuje mechanismus odložení mutujících požadavků při výpadku připojení. Při každém volání funkce `request()` je nejprve zkontrolován příznak `isOffline` ze sdíleného modulu `network-state.ts`. Je-li zařízení offline, požadavek není odeslán, ale serializován a přiložen do fronty uložené v MMKV pod klíčem `offline-queue`. Duplicitní záznamy jsou odfiltrována porovnáním cesty a těla požadavku.

Stav sítě je monitorován komponentou `NetworkProvider` prostřednictvím funkce `addNetworkStateL` z knihovny `expo-network`. Při obnovení připojení je zavolána funkce `flushQueue()`, která sekvenčně přehraje uložené požadavky a neúspěšné ponechá ve frontě pro příští pokus. Souběžně je emitována událost `emitReconnect()`, na kterou jsou přihlášeny všechny doménové Context Providers a reagují opětovným načtením dat ze serveru. Podrobný popis algoritmů `request()` a `flushQueue()` je uveden v sekci 2.5.5.

Lokální oznámení

Plánování oznámení je centralizováno do hooku `useNotifications()` (`src/lib/notifications.ts`). Funkce `scheduleNotification()` přijímá identifikátor, titulek, text, čas a příznak opakování. Opakující se oznámení (připomenutí spánku, vstávání, cvičení) jsou naplánována s triggerem typu `DAILY` zadáváním hodiny a minuty; jednorázová s triggerem `TIME_INTERVAL`. Stav naplánovaných oznámení je synchronizován s MMKV, aby byl dostupný i bez živého systémového volání. Při globálním vypnutí oznámení jsou zrušena všechna naplánovaná upozornění.

2.3.2 Serverová část

Middlewarový řetězec

Každý příchozí HTTP požadavek prochází middlewarovým řetězcem definovaným v `src/server.ts`. Jako první je aplikován middleware `secureHeaders()` nastavující bezpečnostní HTTP hlavičky (například `X-Content-Type-Options`, `X-Frame-Options`), následuje `trimTrailingSlash()`

pro normalizaci cest URL a `requestLogger`, který zaznamenává metodu, cestu, stavový kód, dobu zpracování a IP adresu klienta. Na všech cestách je poté injektována informace o relaci voláním `auth.api.getSession()`: výsledek je uložen do kontextu jako `c.get("user")` a `c.get("session")`; pro nepřihlášené požadavky jsou obě hodnoty nastaveny na `null`. Chráněné routy uplatňují middleware `requireAuth()`, který vrací odpověď `401 Unauthorized` při chybějící relaci, resp. `requireAdmin()`, jenž navíc ověřuje roli `"admin"` v poli role uživatele.

Struktura endpointů

Routy jsou organizovány do čtyř doménových skupin pod základní cestou `/api`. Skupina `/api/auth/*` je delegována přímo na handler knihovny Better Auth a nevyžaduje autentizaci. Skupina `/api/workout/*` vystavuje veřejně dostupná referenční data (cviky, styly a typy sérií, váhové možnosti). Skupina `/api/food` poskytuje stránkovaný seznam potravin a fulltextové vyhledávání (parametr `name` využívající `pg_trgm` [21], parametr `gtin` pro vyhledávání dle čárového kódu). Skupina `/api/user/*` sdružuje chráněné endpointy pro spánek (`/sleep`), výživu (`/food`), tréninky (`/workout`, `/workout/template`) a uživatelský profil (`/data`). Schémata požadavků a odpovědí jsou validována knihovnou Zod a dokumentována prostřednictvím integrace `@hono/zod-openapi` přístupné na `/api/ui` ve vývojovém prostředí.

Nasazení a reverzní proxy

Serverová část je provozována na instanci Oracle Cloud Infrastructure s architekturou ARM. Kontejnerizační soubor `docker-compose.yml` definuje dvě služby: `db` (PostgreSQL 17 s pojmenovaným svazkem `pgdata` pro perzistenci dat) a `api` (sestavení ze zdrojového kódu adresáře `Server/`, mapování portu 3000 a sdílený svazek pro výstup logovacích souborů). Docker Compose [20] běží přímo na VM; reverzní proxy Caddy je nainstalována odděleně jako systémová služba a spravuje ukončení TLS pro domény `lifemeter.fit` a `www.lifemeter.fit`.

Caddy je nakonfigurována se dvěma pravidly omezení frekvence požadavků (*rate limiting*) rozlišujícími dvě časová okna: maximálně 10 požadavků za sekundu a 100 požadavků za minutu, obojí klíčované IP adresou klienta (`key {remote_host}`). Veškeré požadavky nesměřující na cestu `/api` nebo její podstromy jsou okamžitě odmítnuty direktivou `abort`, čímž je server vystaven výhradně definovanému API. Vyhovující požadavky jsou předávány na interní port 3000 aplikačního kontejneru direktivou `reverse_proxy localhost:3000`.

2.3.3 CI/CD pipeline

Průběžné nasazení serverové části je zajištěno workflow GitHub Actions (`.github/workflows/deploy.y` Workflow se spouští automaticky při každém přímém commitu nebo sloučení pull requestu do větve `main`, pokud změna zasahuje do adresáře `Server/`. Jediný krok *Deploy to Oracle VM* využívá akce `appleboy/ssh-action`, která se připojí na produkční server prostřednictvím SSH klíče uloženého v tajných proměnných repozitáře (`SSH_HOST`, `SSH_USER`, `SSH_KEY`). Na serveru jsou poté provedeny příkazy `git pull origin main`, `docker compose down` a `docker compose up -d -build`, čímž dochází k sestavení aktualizované Docker image a opětovnému spuštění služeb s minimálním výpadkem. Databáze zůstává v provozu díky pojmenovanému svazku `pgdata`, který není součástí životního cyklu kontejnerů.

2.4 Návrh aplikace

Návrh aplikace LifeMeter byl proveden před zahájením implementace a sloužil jako závazný rámec pro vývoj. Výsledkem návrhu jsou tři vzájemně doplňující se modely: diagram případů užití popisující funkcionalitu z pohledu uživatele, diagram navigace zachycující strukturu obrazovek a přechody mezi nimi a diagram tříd dokumentující architekturu doménových kontextů klientské vrstvy.

2.4.1 Případy užití

V systému LifeMeter jsou definováni tři aktéři. *Nepřihlášený uživatel* může provést registraci nebo přihlášení. *Přihlášený uživatel* má přístup ke všem funkcím aplikace včetně správy profilu, záznamu spánku, výživy a tréninků a zobrazení domovského přehledu. Externím aktérem je systémová platforma *Health Connect* (Android) resp. *HealthKit* (iOS), se kterou aplikace synchronizuje zdravotní data.

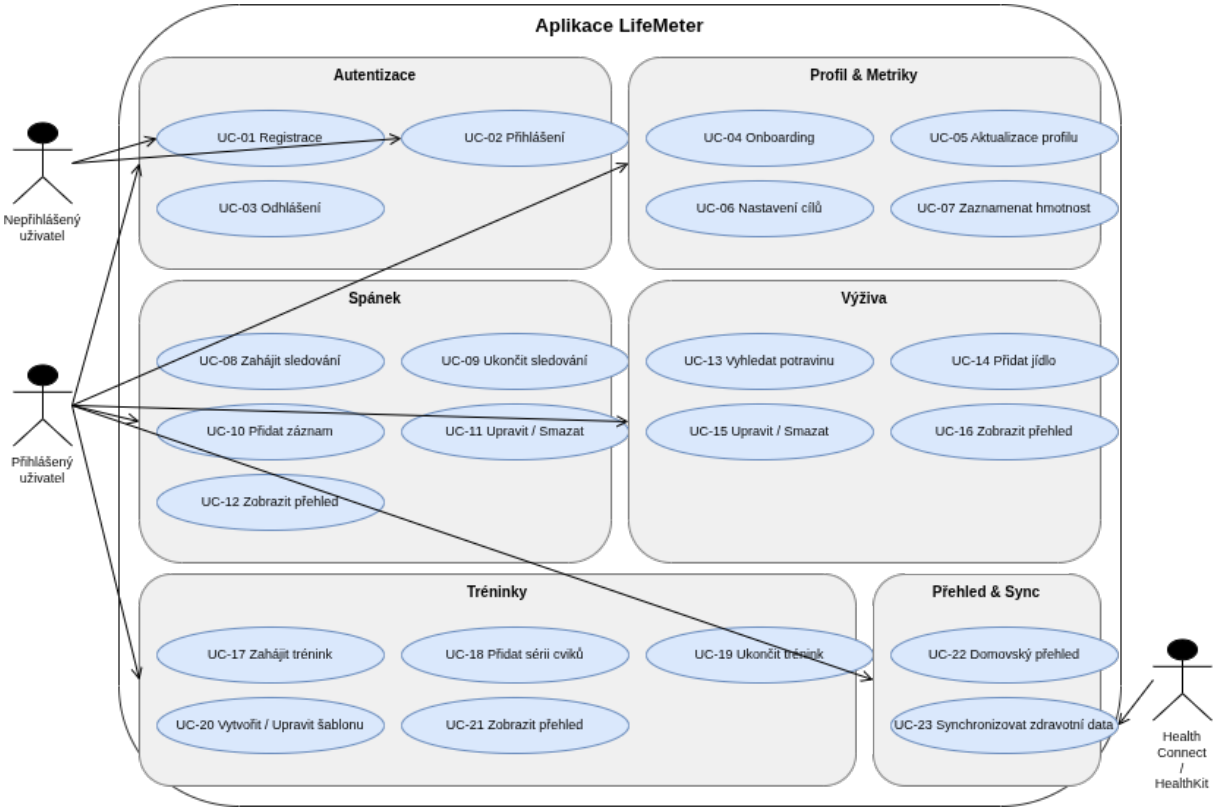
Identifikované případy užití jsou znázorněny na obrázku 2.3

2.4.2 Diagram navigace

Navigační struktura aplikace je rozdělena do dvou zásobníků obrazovek (*stack navigátorů*), jejichž zobrazení závisí na stavu autentizace a existenci uživatelského profilu (obrázek 2.4).

OnboardingStack je prezentován nepřihlášeným uživatelům a obsahuje uvítací obrazovku (*Title*), formulář přihlášení (*SignIn*) a registrace (*SignUp*). Po úspěšné autentizaci je zkontrolována existence profilu: pokud profil neexistuje nebo nebyl dokončen onboarding, je uživatel přesměrován na obrazovku *OnboardingInfoScreen*.

AppStack je zpřístupněn přihlášeným uživatelům s dokončeným profilem. Výchozím prvkem je *AppTabs* – zásobník se spodní navigační lištou obsahující čtyři záložky:



Obrázek 2.3: Diagram případů užití aplikace LifeMeter

Home, *Training*, *Nutrition* a *Sleep*. Nad tímto zásobníkem jsou nadřazeny dvě modální obrazovky: *ActiveWorkout* (aktivní trénink v reálném čase) a *TemplateBuilder* (editor šablony tréninku), přístupné z obrazovky *Training*.

Správa přihlašovacího stavu je zajišťována komponentou `AuthProvider`, která obaluje celý strom aplikace a poskytuje informaci o aktuálním uživateli prostřednictvím React Context API.

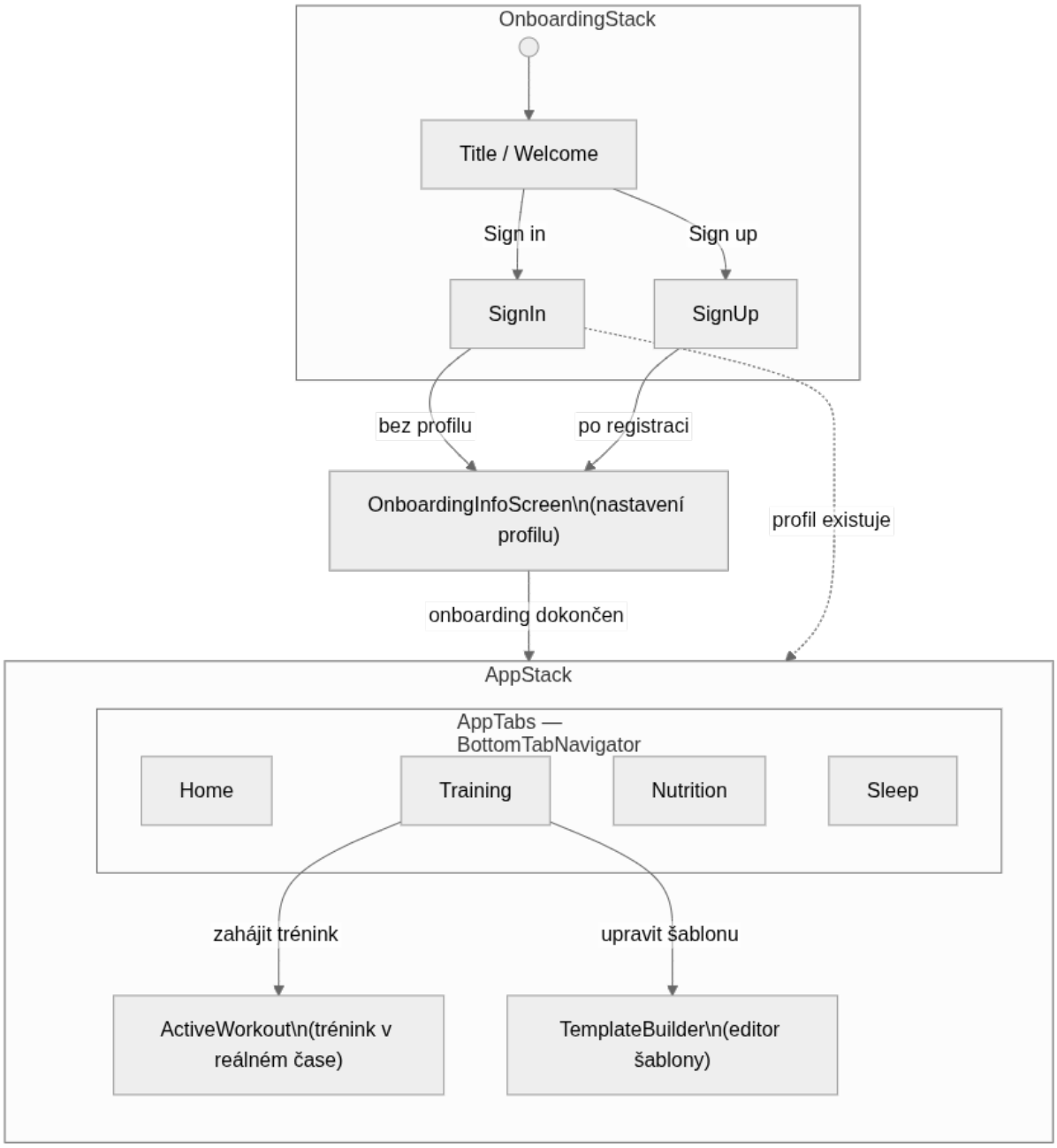
2.4.3 Diagram tříd

Správa aplikačního stavu klientské vrstvy je realizována čtveřicí doménových rozhraní (*Context Providers*), z nichž každý zapouzdřuje data a operace jedné funkční domény (obrázek 2.5).

`SleepStoreContext` spravuje kolekci spánkových záznamů a zpřístupňuje operace pro jejich vytváření, editaci a mazání včetně detekce probíhající spánkové relace (`ongoingSleepSession`).

`NutritionStoreContext` uchovává jídelní záznamy ve struktuře `{userMeal, userFoods}[]` a poskytuje operace CRUD nad jídly a jídelními deníky.

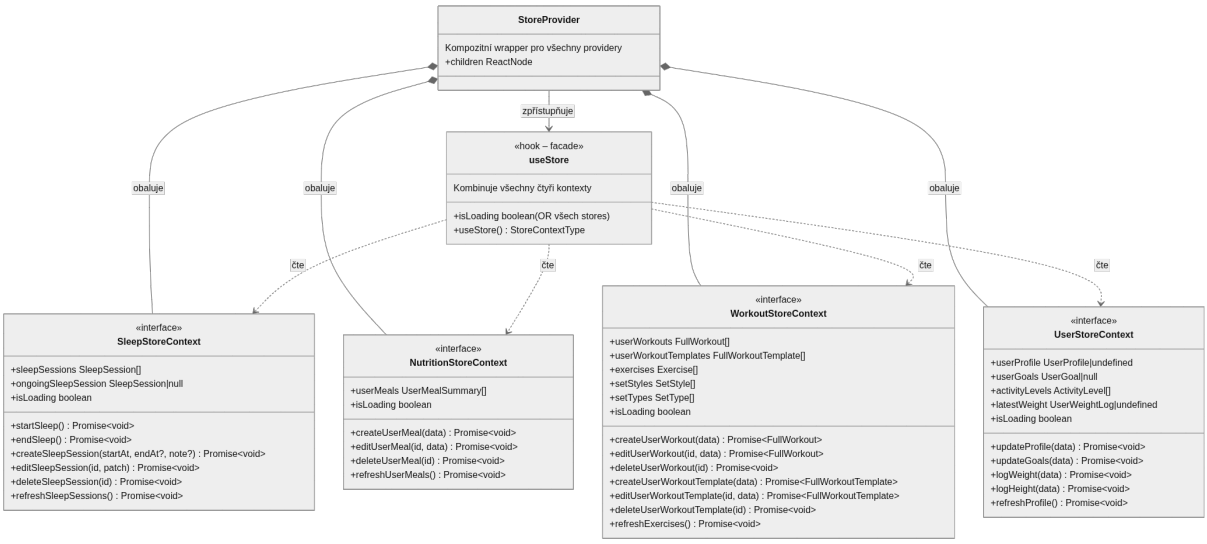
`WorkoutStoreContext` spravuje zaznamenané tréninky, šablony tréninků a referenční číselníky (cviky, styly a typy sérií). Jeho rozhraní je nejrozsáhlejší a odráží komplexnost tréninkové domény.



Obrázek 2.4: Diagram navigační struktury aplikace LifeMeter

`UserStoreContext` poskytuje přístup k uživatelskému profilu, stanoveným cílům a protokolárním záznamům tělesných metrik (hmotnost, výška).

Všechny čtyři providery jsou kompozičně zahrnuty v komponentě `StoreProvider`, která je vložena do kořene stromu aplikace. Pro zpětnou kompatibilitu a pohodlné použití v místech, kde je potřeba přistupovat k více doménám současně, je k dispozici fasádní hook `useStore()`, který sloučí výstupy všech čtyř kontextů do jediné struktury `StoreContextType`. Jeho použití však způsobuje překreslení komponenty při změně v *jakékoliv* doméně; z toho důvodu je doporučeno v nových komponentách preferovat granulární hooky `useSleepStore()`, `useNutritionStore()`, `useWorkoutStore()` a `useUserStore()`.



Obrázek 2.5: Diagram tříd – doménové Context Providers klientské vrstvy

2.5 Popis důležitých algoritmů

2.5.1 Výpočet metabolismu a denního energetického výdeje

Bazální metabolismus (*Basal Metabolic Rate*, BMR) vyjadřuje minimální množství energie potřebné k udržení základních životních funkcí organismu v klidovém stavu. V aplikaci LifeMeter je BMR vypočítán rovnicí Mifflin–St. Jeor [23], která je v současné klinické praxi považována za nejpřesnější predikci klidového energetického výdeje pro obecnou populaci.

Pro pohlaví *muž* platí:

$$\mathrm{BMR}_{\text{muž}} = 10 \cdot m + 6{,}25 \cdot h - 5 \cdot a + 5 \tag{2.1}$$

Pro pohlaví *žena* platí:

$$\mathrm{BMR}_{\text{žena}} = 10 \cdot m + 6{,}25 \cdot h - 5 \cdot a - 161 \tag{2.2}$$

kde m je tělesná hmotnost v kilogramech, h je výška v centimetrech a a je věk v letech. Vstupní hodnoty zadané v jednotkách lbs, st nebo ft jsou před dosazením převedeny na soustavu SI.

Celkový denní energetický výdej (*Total Daily Energy Expenditure*, TDEE) je získán vynásobením BMR koeficientem fyzické aktivity f:

$$\mathrm{TDEE} = \mathrm{BMR} \cdot f \tag{2.3}$$

Koeficient f je uživatelem volen při onboardingu na základě úrovně pohybové aktivity; každá úroveň je definována rozsahem $[f_{\min},\ f_{\max}]$ a ve výpočtu je použita střední hodnota $f = (f_{\min} + f_{\max})/2$.

2.5.2 Stanovení cílového denního kalorického příjmu

Pokud uživatel v průvodci nastavením zadá cílovou hmotnost a cílové datum, je vypočítán upravený denní kalorický příjem tak, aby bylo cíle dosaženo v požadovaném čase. Výpočet vychází z energetické ekvivalence tukové tkáně, kde změna 1 kg tělesné hmotnosti odpovídá přibližně 7 700 kcal [24].

Celková kalorická bilance potřebná ke změně hmotnosti o Δm kilogramů za d dní:

$$\Delta E = \Delta m \cdot 7700 \quad [\mathrm{kcal}] \tag{2.4}$$

Denní kalorická korekce:

$$\delta = \frac{\Delta E}{d} \tag{2.5}$$

Cílový denní příjem:

$$C_{\text{cíl}} = \mathrm{TDEE} + \delta \tag{2.6}$$

Hodnota $\delta > 0$ odpovídá kalorickému přebytku (nabírání hmotnosti), $\delta < 0$ kalorickému deficitu (hubnutí). Pokud uživatel cílovou hmotnost nezadá, je jako cílový příjem použito přímo TDEE.

2.5.3 Výpočet nutričních hodnot jídla

Každá potravina v databázi USDA FoodData Central [22] obsahuje nutriční hodnoty vztažené na 100 g jedlého podílu. Při záznamu jídla uživatel zadá skutečnou gramáž porce g; obsah živiny N pro danou potravinu je pak škálován proporčním faktorem:

$$N_{\text{porce}} = N_{100} \cdot \frac{g}{100} \tag{2.7}$$

Celkový kalorický obsah jídla je stanoven na základě Atwaterových energetických koeficientů [25]:

$$E = 4 \cdot P + 4 \cdot S + 9 \cdot T \quad [\text{kcal}] \tag{2.8}$$

kde P je obsah bílkovin, S obsah sacharidů a T obsah tuků v gramech. Živiny jsou identifikovány prostřednictvím standardizovaných kódů USDA: nutrient_nbr 208 (energie), 203 (bílkoviny), 205 (sacharidy), 204 (tuky).

Pokud uživatel nestanoví explicitní cíle pro jednotlivé makronutrienty, jsou tyto automaticky odvozeny z TDEE s výchozím rozložením 30 % bílkoviny, 40 % sacharidy a 30 % tuky:

$$P_{\text{cíl}} = \left\lfloor \frac{\text{TDEE} \cdot 0{,}30}{4} \right\rfloor \tag{2.9}$$

$$S_{\text{cíl}} = \left\lfloor \frac{\text{TDEE} \cdot 0{,}40}{4} \right\rfloor \tag{2.10}$$

$$T_{\text{cíl}} = \left\lfloor \frac{\text{TDEE} \cdot 0{,}30}{9} \right\rfloor \tag{2.11}$$

2.5.4 Fulltextové vyhledávání potravin

Databáze potravin čítá přes milion záznamů z datového zdroje USDA FoodData Central [22]. Efektivní vyhledávání je zajištěno kombinací PostgreSQL full-text search a GIN indexu. Sloupec `search_vector` je persistován jako generovaný sloupcový `tsvector`:

```
ALTER TABLE food
  ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('english', description)) STORED;

CREATE INDEX idx_food_search_vector
  ON food USING GIN (search_vector);
```

Listing 2.1: Definice generovaného sloupce a GIN indexu

Vstupní dotaz je na straně serveru sanitizován – jsou odstraněny speciální znaky operátorů PostgreSQL (`&`, `|`, `!`, závorky apod.) a každý token je rozšířen o prefixový operátor `:*`, který zajišťuje, že dotaz „appl“ nalezne i záznamy obsahující „apple“ nebo „applesauce“. Jednotlivé tokeny jsou spojeny operátorem AND:

```
vstup:  "whole grain bread"
vystup: "whole:* & grain:* & bread:*"
```

Listing 2.2: Transformace vstupního řetězce na tsquery

Výsledky jsou řazeny víceúrovňovým klíčem tak, aby byly upřednostněny stručné a přesné shody před delšími nebo méně relevantními záznamy:

1. přesná shoda s dotazem (`ILIKE $query`) – priorita 0,
2. prefixová shoda (`ILIKE $query||'%'`) – priorita 1,
3. ostatní výsledky seřazené dle `ts_rank_cd` a délky popisu – priorita 2.

2.5.5 Offline fronta požadavků

Aplikace implementuje princip *offline-first* [7]: při ztrátě síťového připojení nejsou mutační požadavky (POST, PUT, DELETE) zahozeny, nýbrž jsou přidány do fronty v lokálním úložišti MMKV [8] a odeslány po obnovení připojení. Čtecí požadavky (GET, HEAD) jsou vždy odesílány okamžitě a do fronty zařazeny nejsou.

```
FUNCTION request(path, init) -> T | undefined
  IF isOffline AND method(init) ≠ GET AND offlineModeEnabled THEN
    IF NOT queue.contains(path, init) THEN   -- deduplikace
      queue.push(path, init)                 -- persistuj do MMKV
    END IF
    RETURN undefined                         -- optimisticky navrat
  END IF

  controller ← AbortController(timeout = 5000 ms)
  response   ← HTTP.send(path, init, signal = controller)
  RETURN parse(response)
END FUNCTION

FUNCTION flushQueue() -> void
  pending ← queue.readAll()
  queue.clear()                              -- smaz pred odesilani

  FOR EACH (path, init) IN pending DO
    IF request(path, init) FAILS THEN
      queue.push(path, init)                 -- vrat zpet pri selhani
    END IF
  END FOR
END FUNCTION
```

Listing 2.3: Pseudokód zpracování HTTP požadavku s offline frontou

Deduplikace je zajištěna porovnáním serializované cesty a těla požadavku před vložením do fronty, čímž je zamezeno opakovanému záznamu totožné operace.

Každý HTTP požadavek je opatřen timeoutem 5 000 ms prostřednictvím `AbortController`, čímž je zamezeno blokování při nedostupném serveru.

Výsledky

Diskuze

Závěr

Literatura

[1] M. Swan. „The Quantified Self: Fundamental Disruption in Big Data Science and Biological Discovery“. In: *Big Data* 1.2 (čvn. 2013). PMID: 27442063, s. 85–99. ISSN: 2167-6461. DOI: 10.1089/big.2012.0002. URL: https://journals.sagepub.com/doi/10.1089/big.2012.0002 (cit. 11. 03. 2026).

[2] Meta Platforms, Inc. *React Native Documentation*. 2026. URL: https://reactnative.dev/docs/getting-started (cit. 12. 03. 2026).

[3] 650 Industries, Inc. *Expo Documentation*. 2026. URL: https://docs.expo.dev/ (cit. 12. 03. 2026).

[4] Jarred Sumner. *Bun Documentation*. 2026. URL: https://bun.sh/docs (cit. 12. 03. 2026).

[5] Google LLC. *Health Connect – Android Developers Documentation*. 2024. URL: https://developer.android.com/health-and-fitness/guides/health-connect (cit. 15. 03. 2026).

[6] Apple Inc. *HealthKit – Apple Developer Documentation*. 2024. URL: https://developer.apple.com/documentation/healthkit (cit. 15. 03. 2026).

[7] Gregor Hoff a Martin Kleppmann. *Offline First – Designing for Unreliable Networks*. 2015. URL: https://offlinefirst.org/ (cit. 15. 03. 2026).

[8] Tencent. *MMKV – Fast, Persistent Key-Value Storage*. 2024. URL: https://github.com/Tencent/MMKV (cit. 15. 03. 2026).

[9] Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm (cit. 15. 03. 2026).

[10] OWASP Foundation. *OWASP Mobile Application Security Verification Standard*. 2024. URL: https://mas.owasp.org/MASVS/ (cit. 15. 03. 2026).

[11] Google LLC. *Google Fit: Activity Tracking*. 2024. URL: https://www.google.com/fit/ (cit. 15. 03. 2026).

[12] Apple Inc. *Apple Health*. 2024. URL: https://www.apple.com/health/ (cit. 15. 03. 2026).

[13] MyFitnessPal, Inc. *MyFitnessPal: Calorie Counter and Diet Tracker*. 2024. URL: https://www.myfitnesspal.com/ (cit. 15. 03. 2026).

[14] Cronometer Software Inc. *Cronometer: Track Nutrition & Count Calories*. 2024. URL: https://cronometer.com/ (cit. 15. 03. 2026).

[15] Sleep Cycle AB. *Sleep Cycle: Sleep Tracker*. 2024. URL: https://www.sleepcycle.com/ (cit. 15. 03. 2026).

[16] Yusuke Wada. *Hono Documentation*. 2024. URL: https://hono.dev/docs/ (cit. 15. 03. 2026).

[17] Paul Bratslavsky. *Bun vs Node.js: A Practical Comparison Every Developer Should Read*. 2025. URL: https://strapi.io/blog/bun-vs-nodejs-performance-comparison-guide (cit. 16. 03. 2026).

[18] The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*. 2024. URL: https://www.postgresql.org/docs/17/ (cit. 15. 03. 2026).

[19] Better Auth Contributors. *Better Auth Documentation*. 2024. URL: https://www.better-auth.com/docs (cit. 15. 03. 2026).

[20] Docker, Inc. *Docker Compose Documentation*. 2024. URL: https://docs.docker.com/compose/ (cit. 15. 03. 2026).

[21] The PostgreSQL Global Development Group. *pg_trgm – PostgreSQL Extension for Trigram Matching*. 2024. URL: https://www.postgresql.org/docs/17/pgtrgm.html (cit. 15. 03. 2026).

[22] U.S. Department of Agriculture. *FoodData Central*. 2024. URL: https://fdc.nal.usda.gov/ (cit. 15. 03. 2026).

[23] Mark D. Mifflin et al. „A new predictive equation for resting energy expenditure in healthy individuals“. In: *The American Journal of Clinical Nutrition* 51.2 (1990), s. 241–247. DOI: 10.1093/ajcn/51.2.241.

[24] Kevin D. Hall. „What is the required energy deficit per unit weight loss?“ In: *International Journal of Obesity* 32.3 (2008), s. 573–576. DOI: 10.1038/sj.ijo.0803720.

[25] Wilbur O. Atwater. *Principles of Nutrition and Nutritive Value of Food*. United States Department of Agriculture, 1902. URL: https://archive.org/details/principlesofnutr00atwa (cit. 18. 03. 2026).

Seznam obrázků

2.1 Přehled architektury systému LifeMeter 14
2.2 Entity-relationship diagram databázového schématu 16
2.3 Diagram případů užití aplikace LifeMeter 25
2.4 Diagram navigační struktury aplikace LifeMeter 26
2.5 Diagram tříd – doménové Context Providers klientské vrstvy 27

Seznam tabulek

2.1 Tabulka `user` – uživatelský účet 15
2.2 Tabulka `food` – potravina 17
2.3 Tabulka `nutrient_value` – nutriční hodnota potraviny 17
2.4 Tabulka `portion` – porce potraviny 17
2.5 Tabulka `user_meal` – jídlo uživatele 18
2.6 Tabulka `user_food` – položka jídla 18
2.7 Tabulka `user_sleep` – spánková seance 18
2.8 Tabulka `workout_template` – šablona tréninku 18
2.9 Tabulka `template_workout_set` – série v šabloně tréninku 19
2.10 Tabulka `workout` – zaznamenaný trénink 19
2.11 Tabulka `workout_set` – zaznamenaná série tréninku 19
2.12 Tabulka `user_profile` – profil uživatele 20
2.13 Tabulka `user_goal` – cíle uživatele 20
2.14 Tabulka `user_weight_log` – záznam hmotnosti 21

Seznam příloh

A Diagramy

B Tabulky

C Schémata

D API dokumentace

E Instalační návod